

ТЕМА: МАССИВЫ И РАБОТА С НИМИ. БИНАРНЫЙ ПОИСК



ВВЕДЕНИЕ

При работе с алгоритмами требуется понимать как работает и устроена память в машинах, также как работает динамический массив и как создавать бинарный поиск. В этой теме именно этого вы и узнаете

УРОК: БИНАРНЫЙ ПОИСК - РЕАЛИЗАЦИЯ ПО ИНДЕКСАМ

Бинарный поиск

это эффективный алгоритм поиска элемента в отсортированном массиве, использующий принцип деления пополам



Алгоритм

- 01 Определите начальный (low) и конечный (high) индексы массива
- 02 Найдите средний индекс (mid) и сравните элемент в этом индексе с искомым значением
- 03 Если элемент равен искомому, верните его индекс
- 04 Если искомый элемент меньше, ищите в левой половине массива
- 05 Если искомый элемент больше, ищите в правой половине массива
- 06 Повторяйте до тех пор, пока low не станет больше high

Пример кода

```
def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1 # Если элемент не найден

# Пример использования:
arr = [10, 20, 30, 40, 50]
target = 30
print("Индекс элемента:", binary_search(arr, target))
```

Преимущества

Эффективность поиска в отсортированных структурах данных

Значительное сокращение времени поиска по сравнению с линейным поиском

Сложность

Временная сложность: $O(\log n)$, где n — количество элементов в массиве

Пространственная сложность: $O(1)$, так как используется фиксированное количество дополнительной памяти

ВЫВОД

Мы рассмотрели основы работы с массивами. Эти концепции являются основой для эффективного управления данными и поиска информации в программировании